

Tema 5 — Polimorfismo

1) Qué es polimorfismo y para qué sirve

El polimorfismo es la capacidad de tratar objetos de distintos tipos de forma uniforme a través de una referencia común (normalmente el supertipo).

Se apoya en la relación de herencia “es-un”: si Artillero y Zapador son subtipos de Soldado, pueden usarse donde se espere Soldado.

Utilidad: escribir código más general, reusable y extensible (por ejemplo, recorrer un array de Soldado y llamar saludar() sin distinguir tipos). El comportamiento concreto se decide en ejecución.

2) Sobreescritura (override): base del polimorfismo

La sobreescritura es cuando una subclase redefine un método de la superclase manteniendo la misma firma (mismo nombre y parámetros).

Cuando se llama al método mediante una referencia del supertipo, se ejecuta la versión correspondiente al tipo real del objeto.

Ejemplo conceptual del material: Soldado.saludar() y Zapador lo redefine.

3) Ligadura dinámica (enlace tardío) y relación con polimorfismo

Ligadura dinámica = resolver qué método se ejecuta en tiempo de ejecución (no en compilación), según el tipo real del objeto.

El polimorfismo “necesita” esa ligadura dinámica para que la misma llamada (s.saludar()) ejecute implementaciones distintas según el objeto.

Comparación de lenguajes

*Java: la ligadura dinámica en métodos de instancia es el comportamiento por defecto.

*C++: hay que marcar métodos como virtual en la base para activar ligadura dinámica; si no, es estática.

*Python: la resolución es dinámica de forma natural, no hay que declarar nada especial.

4) Reutilizar método base al sobrescribir: super

Al sobrescribir, puedes invocar la implementación base y luego añadir comportamiento. En Java se usa super.

Ej.: Zapador.saludar() llama a super.saludar() y después añade un mensaje extra.

5) Restricciones en Java al sobrescribir + diferencia con sobrecarga

Al sobrescribir en Java:

- *Debe mantenerse la misma firma (mismos parámetros).

- *El retorno debe ser el mismo o covariante (subtipo).

- *No se puede reducir visibilidad (ej. public → protected).

- *Excepciones: no declarar excepciones más generales que las del método base (sí iguales o más específicas).

Overriding vs Overloading:

Overriding: entre superclase y subclase, redefine un método, se decide en ejecución (polimorfismo).

Overloading: mismo nombre pero distinta lista de parámetros; se resuelve en compilación.

@Override

No es obligatoria, pero recomendable: fuerza al compilador a comprobar que realmente estás sobrescribiendo (evita errores por firma incorrecta).

6) Polimorfismo “desde el principio” en Java: Object

En Java se usa polimorfismo muy pronto: al redefinir toString() o equals() ya estás aprovechando el mecanismo (una referencia Object llamando a toString() ejecuta la versión del tipo real).

7) Clases abstractas y métodos abstractos

Clase abstracta: no se usa para instanciar directamente; sirve de base y puede tener métodos implementados y no implementados. Se declara con abstract.

Método abstracto: no tiene cuerpo e “obliga” a subclases a implementarlo. Solo aparece dentro de una clase abstracta.

No se pueden crear instancias de una clase abstracta porque está incompleta.

Ejemplo del material: Soldado abstracto con atacar() abstracto, implementado por Artillero y Zapador.

8) final en clases y métodos (relación con polimorfismo)

En Java:

*final en un método: no se puede sobrescribir.

*final en una clase: no se puede heredar de ella.

Relación: limita el polimorfismo basado en herencia, porque impide crear subtipos o redefinir comportamiento.

9) Interfaces en Java: qué son y cuántas se pueden implementar

Las interfaces definen un “contrato” de métodos que una clase debe implementar.

Se parecen a clases abstractas en que pueden exigir métodos sin implementación, pero se presentan como mecanismo para definir capacidades y permitir “combinación” de comportamientos.

Una clase puede implementar más de una interfaz (forma típica de “herencia múltiple” de comportamiento en Java).

10) Herencia de interfaces (y herencia múltiple de interfaces)

Una interfaz puede extender otra, heredando sus métodos.

Además, existe herencia múltiple de interfaces (una interfaz puede extender varias).

11) Polimorfismo con jerarquías abstractas + instanceof + downcasting (ejemplo Punto)

Se propone modelar Punto como clase abstracta con calcularDistanciaA(Punto) abstracto, e implementarlo en Punto2D y Punto3D.

Para asegurar distancias solo entre puntos del mismo subtipo, se usa instanceof y downcasting cuando proceda.

Se aplica la idea con una clase Linea que guarda dos Punto sin saber si son 2D o 3D; calcula su longitud delegando en calcularDistanciaA. Así Linea funciona sin conocer dimensiones